

```
/*fun main() {  
  
    val morningNotification = 51  
  
    val eveningNotification = 135  
  
    printNotificationSummary(morningNotification)  
    printNotificationSummary(eveningNotification)  
}
```

```
fun printNotificationSummary(numberOfMessages: Int) {  
    if (numberOfMessages < 100) {  
        println("Tienes ${numberOfMessages} notificaciones.")  
    } else {  
        println("Tu teléfono está saturado! Tienes más de 99 notificaciones..")  
    }  
}
```

```
fun main() {  
  
    val child = 5  
  
    val adult = 28  
  
    val senior = 87  
  
  
    val isMonday = true  
  
  
    println("El precio del boleto de cine para una persona de edad $child es  
\\${ticketPrice(child, isMonday)}.")
```

```

    println("El precio del boleto de cine para una persona de edad $adult es
    \${ticketPrice(adult, isMonday)}.")

    println("El precio del boleto de cine para una persona de edad $senior es
    \${ticketPrice(senior, isMonday)}.")
}

```

```

fun ticketPrice(age: Int, isMonday: Boolean): Int {
    return when(age) {
        in 0..12 -> 15
        in 13..60 -> if (isMonday) 25 else 30
        in 61..100 -> 20
        else -> -1
    }
}

```

```

fun main() {
    printFinalTemperature(27.0, "Celsius", "Fahrenheit") { 9.0 / 5.0 * it + 32 }
    printFinalTemperature(350.0, "Kelvin", "Celsius") { it - 273.15 }
    printFinalTemperature(10.0, "Fahrenheit", "Kelvin") { 5.0 / 9.0 * (it - 32) + 273.15 }
}

```

```

fun printFinalTemperature(
    initialMeasurement: Double,
    initialUnit: String,
    finalUnit: String,
    conversionFormula: (Double) -> Double

```

```
) {  
    val finalMeasurement = String.format("%.2f",  
conversionFormula(initialMeasurement)) // two decimal places  
    println("$initialMeasurement grados $initialUnit es $finalMeasurement grados  
$finalUnit.")  
}
```

```
fun main() {  
    val brunoSong = Song("We Don't Talk About Bruno", "Encanto Cast", 2022,  
1_000_000)  
    brunoSong.printDescription()  
    println(brunoSong.isPopular)  
}
```

```
class Song(  
    val title: String,  
    val artist: String,  
    val yearPublished: Int,  
    val playCount: Int  
) {  
    val isPopular: Boolean  
        get() = playCount >= 1000  
  
    fun printDescription() {  
        println("$title, realizado por $artist, fue lanzado en $yearPublished.")  
    }  
}
```

```
}
```

```
fun main() {  
    val amanda = Person("Amanda", 33, "play tennis", null)  
    val atiqah = Person("Atiqah", 28, "climb", amanda)  
  
    amanda.showProfile()  
    atiqah.showProfile()  
}
```

```
class Person(val name: String, val age: Int, val hobby: String?, val referrer: Person?) {  
    fun showProfile() {  
        println("Name: $name")  
        println("Age: $age")  
        if(hobby != null) {  
            print("Likes to $hobby. ")  
        }  
        if(referrer != null) {  
            print("Has a referrer named ${referrer.name}")  
            if(referrer.hobby != null) {  
                print(", who likes to ${referrer.hobby}.")  
            } else {  
                print(".")  
            }  
        } else {  
            print(".")  
        }  
    }  
}
```

```
        print("Doesn't have a referrer.")
    }
    print("\n\n")
}
}
```

```
open class Phone(var isScreenLightOn: Boolean = false){
    open fun switchOn() {
        isScreenLightOn = true
    }
}
```

```
fun switchOff() {
    isScreenLightOn = false
}
```

```
fun checkPhoneScreenLight() {
    val phoneScreenLight = if (isScreenLightOn) "on" else "off"
    println("The phone screen's light is $phoneScreenLight.")
}
}
```

```
class FoldablePhone(var isFolded: Boolean = true): Phone() {
    override fun switchOn() {
        if (!isFolded) {
            isScreenLightOn = true
        }
    }
}
```

```

    }

    fun fold() {
        isFolded = true
    }

    fun unfold() {
        isFolded = false
    }
}

fun main() {
    val newFoldablePhone = FoldablePhone()

    newFoldablePhone.switchOn()
    newFoldablePhone.checkPhoneScreenLight()
    newFoldablePhone.unfold()
    newFoldablePhone.switchOn()
    newFoldablePhone.checkPhoneScreenLight()
}

fun main() {
    val winningBid = Bid(5000, "Private Collector")

    println("Item A is sold at ${auctionPrice(winningBid, 2000)}.")
    println("Item B is sold at ${auctionPrice(null, 3000)}.")
}

```

```
}
```

```
class Bid(val amount: Int, val bidder: String)
```

```
fun auctionPrice(bid: Bid?, minimumPrice: Int): Int {  
    return bid?.amount ?: minimumPrice  
}
```

```
-----
```

```
class FillInTheBlankQuestion(  
  

```

```
)
```

```
class TrueOrFalseQuestion(  
  

```

```
)
```

```
class NumericQuestion(  
  

```

```
)
```

```
class Question()
```

```
class Question<T>(  
  

```

```
    val questionText: String,
```

```
    val answer: T,  
    val difficulty: String  
)
```

```
fun main() {  
    val question1 = Question<String>("Quoth the raven ____", "nevermore", "medium")  
    val question2 = Question<Boolean>("The sky is green. True or false", false, "easy")  
    val question3 = Question<Int>("How many days are there between full moons?", 28,  
    "hard")  
}
```

```
class Question<T>(  
    val questionText: String,  
    val answer: T,  
    val difficulty: Difficulty  
)
```

```
enum class Difficulty {  
    EASY, MEDIUM, HARD  
}
```

```
fun main() {  
    val question1 = Question<String>("Quoth the raven ____", "nevermore",  
    Difficulty.MEDIUM)  
    val question2 = Question<Boolean>("The sky is green. True or false", false,  
    Difficulty.EASY)
```



```
    val question3 = Question<Int>("How many days are there between full moons?", 28,
Difficulty.HARD)

    println(question1.toString())
}
```

```
object StudentProgress {

    var total: Int = 10

    var answered: Int = 3

}
```

```
fun main() {

    println("${StudentProgress.answered} of ${StudentProgress.total} answered.")

}
```

```
class Question<T>(

    val questionText: String,

    val answer: T,

    val difficulty: Difficulty

)
```

```
enum class Difficulty {

    EASY, MEDIUM, HARD

}
```

```

class Quiz {

    val question1 = Question<String>("Quoth the raven __", "nevermore",
Difficulty.MEDIUM)

    val question2 = Question<Boolean>("The sky is green. True or false", false,
Difficulty.EASY)

    val question3 = Question<Int>("How many days are there between full moons?", 28,
Difficulty.HARD)


    object StudentProgress {

        var total: Int = 10

        var answered: Int = 3

    }

}


fun main() {

    println("${Quiz.StudentProgress.answered} of ${Quiz.StudentProgress.total}
answered.")

}


class Question<T>(

    val questionText: String,

    val answer: T,

    val difficulty: Difficulty

)


enum class Difficulty {

```

```

    EASY, MEDIUM, HARD
}

class Quiz {
    val question1 = Question("Quoth the raven __", "nevermore", Difficulty.MEDIUM)
    val question2 = Question("The sky is green. True or false", false, Difficulty.EASY)
    val question3 = Question("How many days are there between full moons?", 28,
        Difficulty.HARD)

    object StudentProgress {
        var total: Int = 10
        var answered: Int = 3

        val progressText: String
            get() = "$answered of $total answered"
    }
}

fun main() {
    println(Quiz.StudentProgress.progressText)
}

class Question<T>(
    val questionText: String,
    val answer: T,
    val difficulty: Difficulty

```

)

```
enum class Difficulty {  
    EASY, MEDIUM, HARD  
}
```

```
interface ProgressPrintable {  
    val progressText: String  
    fun printProgressBar()  
}
```

```
class Quiz : ProgressPrintable {
```

```
    val question1 = Question("Quoth the raven __", "nevermore", Difficulty.MEDIUM)  
    val question2 = Question("The sky is green. True or false", false, Difficulty.EASY)  
    val question3 = Question("How many days are there between full moons?", 28,  
Difficulty.HARD)
```

```
    object StudentProgress {  
        var total: Int = 10  
        var answered: Int = 3  
    }
```

```
    override val progressText: String
```

```
        get() = "${StudentProgress.answered} of ${StudentProgress.total} answered"
```

```

override fun printProgressBar() {
    repeat(StudentProgress.answered) { print("█") }
    repeat(StudentProgress.total - StudentProgress.answered) { print("░") }
    println()
    println(progressText)
}
}

```

```

fun main() {
    Quiz().printProgressBar()
}

```

```

class Question<T>(
    val questionText: String,
    val answer: T,
    val difficulty: Difficulty
)

```

```

enum class Difficulty {
    EASY, MEDIUM, HARD
}

```

```

interface ProgressPrintable {
    val progressText: String
    fun printProgressBar()
}

```

```

class Quiz : ProgressPrintable {

    val question1 = Question("Quoth the raven __", "nevermore", Difficulty.MEDIUM)
    val question2 = Question("The sky is green. True or false", false, Difficulty.EASY)
    val question3 = Question("How many days are there between full moons?", 28,
Difficulty.HARD)

    object StudentProgress {
        var total: Int = 10
        var answered: Int = 3
    }

    override val progressText: String
        get() = "${StudentProgress.answered} of ${StudentProgress.total} answered"

    override fun printProgressBar() {
        repeat(StudentProgress.answered) { print("█") }
        repeat(StudentProgress.total - StudentProgress.answered) { print("░") }
        println()
        println(progressText)
    }

    fun printQuiz() {
        question1.let {
            println(it.questionText)

```

```
        println(it.answer)
        println(it.difficulty)
    }
    println()
```

```
question2.let {
    println(it.questionText)
    println(it.answer)
    println(it.difficulty)
}
println()
```

```
question3.let {
    println(it.questionText)
    println(it.answer)
    println(it.difficulty)
}
println()
}
}
```

```
fun main() {
    Quiz().apply {
        printQuiz()
    }
}
```

```
class Cookie(  
    val name: String,  
    val softBaked: Boolean,  
    val hasFilling: Boolean,  
    val price: Double  
)
```

```
val cookies = listOf(  
    Cookie(  
        name = "Chocolate Chip",  
        softBaked = false,  
        hasFilling = false,  
        price = 1.69  
    ),  
    Cookie(  
        name = "Banana Walnut",  
        softBaked = true,  
        hasFilling = false,  
        price = 1.49  
    ),  
    Cookie(  
        name = "Vanilla Creme",  
        softBaked = false,  
        hasFilling = true,
```



```
    price = 1.59
),
Cookie(
    name = "Chocolate Peanut Butter",
    softBaked = false,
    hasFilling = true,
    price = 1.49
),
Cookie(
    name = "Snickerdoodle",
    softBaked = true,
    hasFilling = false,
    price = 1.39
),
Cookie(
    name = "Blueberry Tart",
    softBaked = true,
    hasFilling = true,
    price = 1.79
),
Cookie(
    name = "Sugar and Sprinkles",
    softBaked = false,
    hasFilling = false,
    price = 1.39
)
```

```
)
```

```
fun main() {  
    cookies.forEach {  
        println("Menu item: $it")  
    }  
}
```

```
class Cookie(  
    val name: String,  
    val softBaked: Boolean,  
    val hasFilling: Boolean,  
    val price: Double  
)
```

```
val cookies = listOf(  
    Cookie("Chocolate Chip", false, false, 1.69),  
    Cookie("Banana Walnut", true, false, 1.49),  
    Cookie("Vanilla Creme", false, true, 1.59),  
    Cookie("Chocolate Peanut Butter", false, true, 1.49),  
    Cookie("Snickerdoodle", true, false, 1.39),  
    Cookie("Blueberry Tart", true, true, 1.79),  
    Cookie("Sugar and Sprinkles", false, false, 1.39)  
)
```

```
fun main() {
```

```

val fullMenu = cookies.map {
    "${it.name} - ${it.price}"
}

println("Full menu:")
fullMenu.forEach {
    println(it)
}
}

class Cookie(
    val name: String,
    val softBaked: Boolean,
    val hasFilling: Boolean,
    val price: Double
)

val cookies = listOf(
    Cookie("Chocolate Chip", false, false, 1.69),
    Cookie("Banana Walnut", true, false, 1.49),
    Cookie("Vanilla Creme", false, true, 1.59),
    Cookie("Chocolate Peanut Butter", false, true, 1.49),
    Cookie("Snickerdoodle", true, false, 1.39),
    Cookie("Blueberry Tart", true, true, 1.79),
    Cookie("Sugar and Sprinkles", false, false, 1.39)
)

```

```
fun main() {

    val fullMenu = cookies.map {
        "${it.name} - ${it.price}"
    }

    println("Full menu:")
    fullMenu.forEach {
        println(it)
    }

    println()

    val softBakedMenu = cookies.filter {
        it.softBaked
    }

    println("Soft cookies:")
    softBakedMenu.forEach {
        println("${it.name} - ${it.price}")
    }
}

class Cookie(
    val name: String,
```

```
    val softBaked: Boolean,  
    val hasFilling: Boolean,  
    val price: Double  
)
```

```
val cookies = listOf(  
    Cookie("Chocolate Chip", false, false, 1.69),  
    Cookie("Banana Walnut", true, false, 1.49),  
    Cookie("Vanilla Creme", false, true, 1.59),  
    Cookie("Chocolate Peanut Butter", false, true, 1.49),  
    Cookie("Snickerdoodle", true, false, 1.39),  
    Cookie("Blueberry Tart", true, true, 1.79),  
    Cookie("Sugar and Sprinkles", false, false, 1.39)  
)
```

```
fun main() {  
  
    val fullMenu = cookies.map {  
        "${it.name} - ${it.price}"  
    }  
  
    println("Full menu:")  
    fullMenu.forEach {  
        println(it)  
    }  
}
```

```
println()
```

```
val groupedMenu = cookies.groupBy { it.softBaked }
```

```
val softBakedMenu = groupedMenu[true] ?: listOf()
```

```
val crunchyMenu = groupedMenu[false] ?: listOf()
```

```
println("Soft cookies:")
```

```
softBakedMenu.forEach {
```

```
    println("${it.name} - ${it.price}")
```

```
}
```

```
println()
```

```
println("Crunchy cookies:")
```

```
crunchyMenu.forEach {
```

```
    println("${it.name} - ${it.price}")
```

```
}
```

```
}
```

```
class Cookie(
```

```
    val name: String,
```

```
    val softBaked: Boolean,
```

```
    val hasFilling: Boolean,
```

```
    val price: Double
```

```
)
```

```
val cookies = listOf(  
    Cookie("Chocolate Chip", false, false, 1.69),  
    Cookie("Banana Walnut", true, false, 1.49),  
    Cookie("Vanilla Creme", false, true, 1.59),  
    Cookie("Chocolate Peanut Butter", false, true, 1.49),  
    Cookie("Snickerdoodle", true, false, 1.39),  
    Cookie("Blueberry Tart", true, true, 1.79),  
    Cookie("Sugar and Sprinkles", false, false, 1.39)  
)
```

```
fun main() {
```

```
    val fullMenu = cookies.map {  
        "${it.name} - ${it.price}"  
    }
```

```
    println("Full menu:")
```

```
    fullMenu.forEach {  
        println(it)  
    }
```

```
    println()
```

```
    val groupedMenu = cookies.groupBy { it.softBaked }
```

```
val softBakedMenu = groupedMenu[true] ?: listOf()
val crunchyMenu = groupedMenu[false] ?: listOf()

println("Soft cookies:")
softBakedMenu.forEach {
    println("${it.name} - ${it.price}")
}

println()

println("Crunchy cookies:")
crunchyMenu.forEach {
    println("${it.name} - ${it.price}")
}

println()

val totalPrice = cookies.fold(0.0) { total, cookie ->
    total + cookie.price
}

println("Total price: ${totalPrice}")
}

class Cookie(
```



```
    val name: String,
    val softBaked: Boolean,
    val hasFilling: Boolean,
    val price: Double
)

val cookies = listOf(
    Cookie("Chocolate Chip", false, false, 1.69),
    Cookie("Banana Walnut", true, false, 1.49),
    Cookie("Vanilla Creme", false, true, 1.59),
    Cookie("Chocolate Peanut Butter", false, true, 1.49),
    Cookie("Snickerdoodle", true, false, 1.39),
    Cookie("Blueberry Tart", true, true, 1.79),
    Cookie("Sugar and Sprinkles", false, false, 1.39)
)
```

```
fun main() {

    val fullMenu = cookies.map {
        "${it.name} - ${it.price}"
    }

    println("Full menu:")
    fullMenu.forEach {
        println(it)
    }
}
```

```
println()
```

```
val groupedMenu = cookies.groupBy { it.softBaked }
```

```
val softBakedMenu = groupedMenu[true] ?: listOf()
```

```
val crunchyMenu = groupedMenu[false] ?: listOf()
```

```
println("Soft cookies:")
```

```
softBakedMenu.forEach {
```

```
    println("${it.name} - ${it.price}")
```

```
}
```

```
println()
```

```
println("Crunchy cookies:")
```

```
crunchyMenu.forEach {
```

```
    println("${it.name} - ${it.price}")
```

```
}
```

```
println()
```

```
val totalPrice = cookies.fold(0.0) { total, cookie ->
```

```
    total + cookie.price
```

```
}
```

```
println("Total price: $$${totalPrice}")
```

```
println()
```

```
val alphabeticalMenu = cookies.sortedBy {  
    it.name  
}
```

```
println("Alphabetical menu:")  
alphabeticalMenu.forEach {  
    println(it.name)  
}  
}
```

```
-----  
*/
```

```
data class Event(  
    val title: String,  
    val description: String? = null,  
    val daypart: Daypart,  
    val durationInMinutes: Int  
)
```

```
enum class Daypart {
```

```
MORNING,  
AFTERNOON,  
EVENING  
}
```

```
val Event.durationOfEvent: String  
  
    get() = if (this.durationInMinutes < 60) {  
        "short"  
    } else {  
        "long"  
    }  
}
```

```
fun main() {  
  
    val event1 = Event("Wake up", "Time to get up", Daypart.MORNING, 0)  
    val event2 = Event("Eat breakfast", daypart = Daypart.MORNING, durationInMinutes  
= 15)  
    val event3 = Event("Learn about Kotlin", daypart = Daypart.AFTERNOON,  
durationInMinutes = 30)  
    val event4 = Event("Practice Compose", daypart = Daypart.AFTERNOON,  
durationInMinutes = 60)  
    val event5 = Event("Watch latest DevBytes video", daypart = Daypart.AFTERNOON,  
durationInMinutes = 10)  
    val event6 = Event("Check out latest Android Jetpack library", daypart =  
Daypart.EVENING, durationInMinutes = 45)  
  
    val events = mutableListOf(event1, event2, event3, event4, event5, event6)
```

```
val shortEvents = events.filter { it.durationInMinutes < 60 }  
println("You have ${shortEvents.size} short events.")  
  
val groupedEvents = events.groupBy { it.daypart }  
groupedEvents.forEach { (daypart, events) ->  
    println("$daypart: ${events.size} events")  
}  
  
println("Last event of the day: ${events.last().title}")  
}
```